Practical No:1

Recursive function.

```
#include <stdio.h>

int fibonacci(int n) {
   if (n <= 1) {
      return n;
   }
   return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
   int n = 7;
   for (int i = 0; i < n; i++) {
      printf("%d ", fibonacci(i));
   }
   return 0;
}
```

Output

0 1 1 2 3 5 8

Non-Recursive Fibonacci Program

```
#include <stdio.h>

int main() {

   int n = 7;

   int a = 0, b = 1, c

for (int i = 0; i < n; i++) {

       printf("%d ", a);

       c = a + b;

       a = b;

       b = c;

   }

  return 0;

}
```

Output

0 1 1 2 3 5 8

Practical No:2

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
};
struct Node* createNode(char ch, int freq) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}
void swap(struct Node** a, struct Node** b) {
    struct Node* temp = *a;
    *a = *b;
    *b = temp;
}
void heapify(struct Node* heap[], int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
```

```
    if (left < n &&
        heap[left]->freq < heap[smallest]->freq)
        smallest = left;
    if (right < n &&
        heap[right]->freq < heap[smallest]->freq)
        smallest = right;
    if (smallest != i) {
        swap(&heap[i], &heap[smallest]);
        heapify(heap, n, smallest);
    }
}
struct Node* extractMin(struct Node* heap[], int* n) {
    struct Node* temp = heap[0];
    heap[0] = heap[*n - 1];
    (*n)--;
    heapify(heap, *n, 0);
    return temp;
}
void insert(struct Node* heap[], int* n, struct Node* node) {
    int i = (*n)++;
    heap[i] = node;
    while (i > 0 &&
           heap[(i - 1) / 2]->freq > heap[i]->freq) {
        swap(&heap[i], &heap[(i - 1) / 2]);
```

```
        i = (i - 1) / 2;
    }
}
struct Node* buildHuffman(char ch[], int freq[], int n) {
    struct Node* heap[100];
    int size = 0;
    for (int i = 0; i < n; i++)
        insert(heap, &size, createNode(ch[i], freq[i]));
    while (size > 1) {
        struct Node* left = extractMin(heap, &size);
        struct Node* right = extractMin(heap, &size);
        struct Node* parent =
            createNode('$', left->freq + right->freq);
        parent->left = left;
        parent->right = right;
        insert(heap, &size, parent);
    }
    return extractMin(heap, &size);
}
void printCodes(struct Node* root, int code[], int top) {
    if (root->left) {
        code[top] = 0;
        printCodes(root->left, code, top + 1);
    }
```

```
    if (root->right) {
        code[top] = 1;
        printCodes(root->right, code, top + 1);
    }
    if (!root->left && !root->right) {
        printf("%c: ", root->ch);
        for (int i = 0; i < top; i++)
            printf("%d", code[i]);
        if (top == 0)
            printf("0");
        printf("\n");
    }
}
int main() {
    char ch[] = {'A', 'B', 'C', 'D', 'E', 'F'};
    int freq[] = {5, 9, 12, 13, 16, 45};
    int n = 6;
    int code[100];
    struct Node* root = buildHuffman(ch, freq, n);
    printf("Huffman Codes:\n");
    printCodes(root, code, 0);
    return 0;
}
```

Output

Huffman Codes:

F: 0

C: 100

D: 101

A: 1100

B: 1101

E: 111

Practical No:3

```
#include <stdio.h>
int main() {
   int n = 3;
   float capacity = 50;
   float weight[] = {10, 20, 30};
   float profit[] = {60, 100, 120};
   float ratio[3], totalProfit = 0;
   for (int i = 0; i < n; i++) {
      ratio[i] = profit[i] / weight[i];
   }
   // Sort items by profit/weight ratio
   for (int i = 0; i < n - 1; i++) {
      for (int j = i + 1; j < n; j++) {
         if (ratio[i] < ratio[j]) {
            float temp;
            temp = ratio[i];
            ratio[i] = ratio[j];
            ratio[j] = temp;
            temp = weight[i];
            weight[i] = weight[j];
            weight[j] = temp;
            temp = profit[i];
            profit[i] = profit[j];
```

```
            profit[j] = temp;
        }
    }
}
// Fill the knapsack
for (int i = 0; i < n; i++) {
    if (capacity >= weight[i]) {
        capacity -= weight[i];
        totalProfit += profit[i];
    } else {
        totalProfit += ratio[i] * capacity;
        capacity = 0;
        break;
    }
}
printf("Maximum Profit = %.2f\n", totalProfit);
return 0;
}
```

Output

Maximum Profit = 240.00

Practical No:4

```
#include <stdio.h>
int max(int a, int b) {
   return (a > b) ? a : b;
}
int main() {
   int n = 3;
   int capacity = 50;
   int weight[] = {10, 20, 30};
   int profit[] = {60, 100, 120};
   int dp[4][51];
   for (int i = 0; i <= n; i++) {
      for (int w = 0; w <= capacity; w++) {
         if (i == 0 || w == 0) {
            dp[i][w] = 0;
         } else if (weight[i - 1] <= w) {
            dp[i][w] = max(
               profit[i - 1] +
               dp[i - 1][w - weight[i - 1]],
               dp[i - 1][w]
            );
         } else {
            dp[i][w] = dp[i - 1][w];
         }
```

```
        }
    }
    printf("Maximum Profit = %d\n",
          dp[n][capacity]);
    return 0;
}
```

Output

Maximum Profit = 220

Practical No:5

```
#include <stdio.h>
#include <stdlib.h>
#define N 4
int board[N][N];
int isSafe(int row, int col) {
   int i, j;
   // Check same column
   for (i = 0; i < row; i++) {
      if (board[i][col] == 1)
         return 0;
   }
   // Check upper-left diagonal
   for (i = row - 1, j = col - 1;
       i >= 0 && j >= 0; i--, j--) {
      if (board[i][j] == 1)
         return 0;
   }
   // Check upper-right diagonal
   for (i = row - 1, j = col + 1;
       i >= 0 && j < N; i--, j++) {
      if (board[i][j] == 1)
         return 0;
   }
```

```
    return 1;
}
int solveNQueens(int row) {
    if (row == N)
        return 1;
    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row][col] = 1;
            if (solveNQueens(row + 1))
                return 1;
            // Backtrack
            board[row][col] = 0;
        }
    }
    return 0;
}
int main() {
    if (solveNQueens(0)) {
        printf("Solution of %d Queens:\n", N);
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                printf("%d ", board[i][j]);
            }
            printf("\n");
```

```
        }
    } else {
        printf("No solution exists.\n");
    }
    return 0;
}
```

Output

Solution of 4 Queens:

0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0